



# Sharing One Big AI Model Across Small Machines

*and fixing the split while it runs*

Most systems decide how to cut the model once. We keep checking, and change the cut whenever the machines change.

Author:      Institution:      Date:

# The machines keep changing

1

## A big model does not fit on one machine

So we cut it into blocks of layers and give one block to each machine. Where we cut is the most important choice we make.

2

## Small machines are not like data centres

Laptops and cheap GPU boxes get hot and slow down, their network speed wanders, and sometimes they switch off.

3

## Other systems cut once and never look again

That cut is right at the start, and gets more and more wrong as the machines change.

What that costs

**2.4x**

slower when one machine gets hot

**4.3x**

slower when the network slows too

# What others have done

Ten papers from 2024 and 2025. Full details on slide 10.

- **EdgeShard (2024)**

Picks where to cut with an exact solver. 50% less delay.

- **Galaxy (2024)**

Splits work inside each layer instead of by layer.

- **Helix (2025)**

Chooses placement and routing together on mixed GPUs.

- **TPI-LLM (2024)**

Clever memory handling so 70B models fit in small RAM.

- **prima.cpp (2025)**

Runs 30-70B models on real home machines.

- **Head-level splitting (2025)**

Cuts into very small pieces and moves them when memory is low.

- **Parallax (2025)**

Serves a model across volunteer machines, no central cluster.

- **Learned splitting (2025)**

Learns where to cut as the wireless signal changes.

- **Survey of the field (2025)**

Lists adapting to changing machines as an open problem.

- **Linux schedulers in eBPF (2025)**

Shows eBPF is now used to control, not only to watch.

**What they all have in common: they measure the machines before the run, and never measure again.**

# The problem

We have  $N$  model layers and  $K$  machines of different speeds. While the model is answering, their speed, their network delay, and whether they are even alive all keep changing. Keep choosing a cut that makes the slowest machine as fast as possible - and never restart anything to do it.

That breaks into four jobs:

- |                                                                                              |                                                                                      |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>1 Measure</b><br>network delay and machine heat, cheaply, without changing the model code | <b>2 Decide</b><br>the best cut, fast enough to redo it often, without flip-flopping |
| <b>3 Act</b><br>move layers between running machines without losing requests                 | <b>4 Recover</b><br>notice a dead machine and share its layers among the rest        |

# What is new in our work

1

## We measure the network inside the kernel

Small eBPF programs read the round-trip time Linux already tracks for every connection. Others guess with timers, or measure only before the run.

2

## We never stop deciding

Watch, decide, act, recover - every 2 seconds, for as long as the system runs. The cut is something we keep adjusting, not a setting we pick once.

3

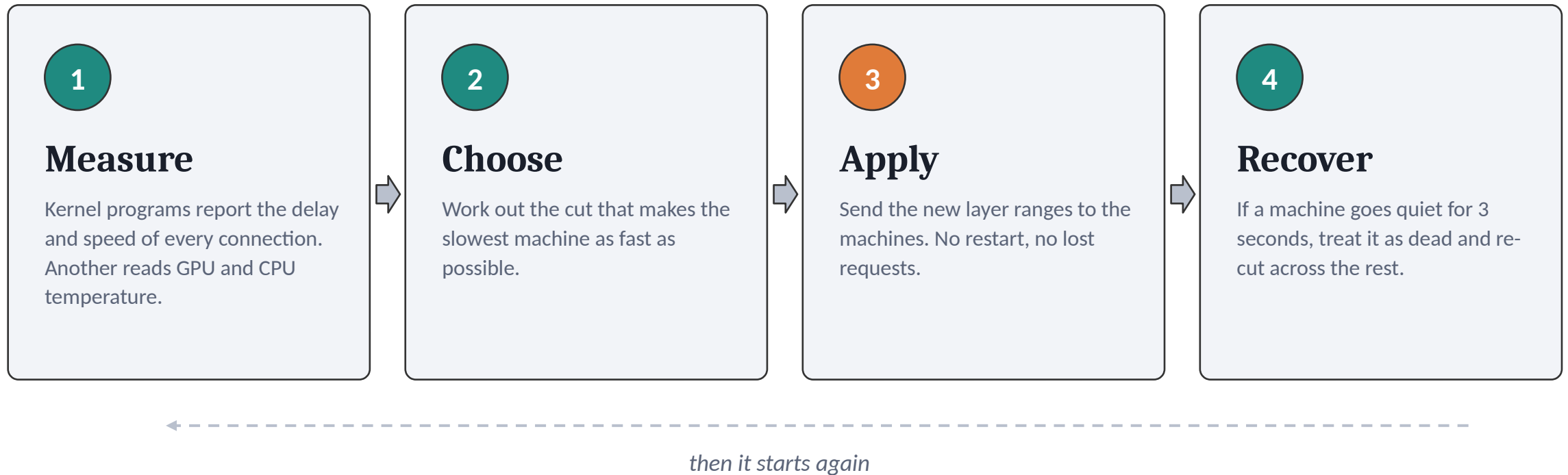
## We add brakes so it does not flip-flop

A new cut is applied only if it is at least 15% better and 30 seconds have passed. The same brake handles heat, network and machine failure.

**We did not invent a new algorithm. We took the standard one and put it inside a loop that never stops measuring.**

# How our system works

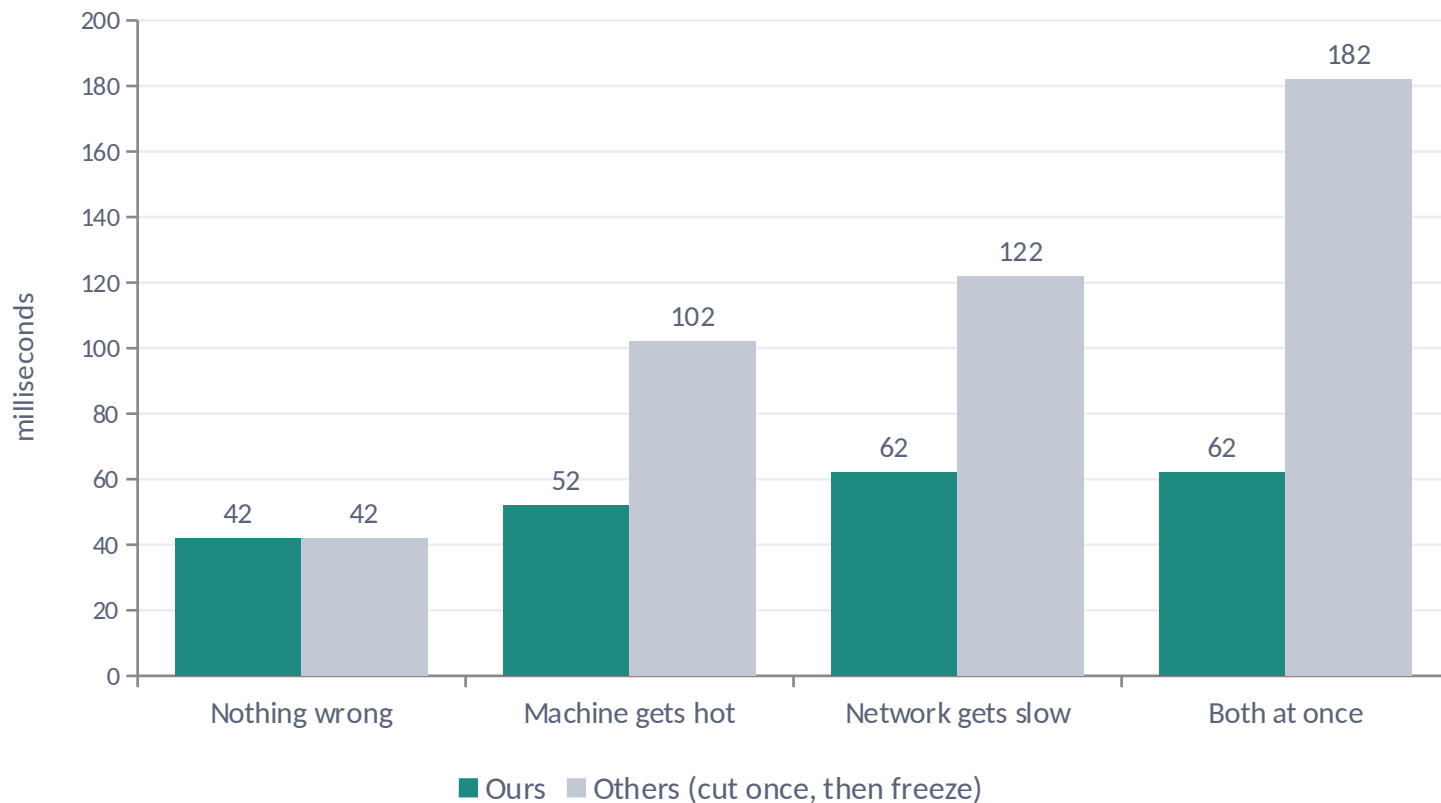
The same four steps run over and over, every 2 seconds.



Every plan carries a version number. Machines refuse messages from an old plan, and because they keep no state, the request is simply replayed.

# Results

How slow the busiest machine gets. Lower is better.



**49%**

better when a machine gets hot

**66%**

better when the network slows too

**It keeps going**

when a machine dies, the others take over.  
A fixed cut simply stops.

# How we compare

|                                 | EdgeShard      | Galaxy         | Helix          | prima.cpp    | Ours                  |
|---------------------------------|----------------|----------------|----------------|--------------|-----------------------|
| Where its numbers come from     | before the run | before the run | before the run | device specs | live, from the kernel |
| Re-cuts while running           | no             | no             | only routing   | no           | every 2 seconds       |
| Reacts to a machine getting hot | no             | no             | no             | no           | yes                   |
| Survives a machine dying        | no             | no             | by copies      | no           | yes, in 3 seconds     |
| Changes without restarting      | no             | no             | no             | no           | yes                   |

We measure how slow the busiest machine gets. The other systems report different measurements, so the numbers are not directly comparable.

## Conclusion

# Where to cut the model is a decision you should keep making, not one you make once.

### 1 Working out the best cut is cheap

It takes microseconds, so there is no reason to do it only once.

### 2 The brakes are what make it usable

One 'must be clearly better' rule and one 'wait a bit' rule handle heat, slow networks and dead machines the same way.

### 3 Measuring in the kernel is free

eBPF sees what really went over the network, not what the program thinks it sent.

# References

[1] EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. M. Zhang, J. Cao, X. Shen, Z. Cui. IEEE Internet of Things Journal, 2024. arXiv:2405.14371.

[2] Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. IEEE INFOCOM, 2024. arXiv:2405.17245.

[3] Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. ACM ASPLOS, 2025. arXiv:2406.01566.

[4] TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices. 2024. arXiv:2410.00531.

[5] Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous and Low-Resource Home Clusters. 2025. arXiv:2504.08791.

[6] Large Language Model Partitioning for Low-Latency Inference at the Edge. 2025. arXiv:2505.02533.

[7] Parallax: Efficient LLM Inference Service over Decentralized Environment. 2025. arXiv:2509.26182.

[8] Adaptive layer splitting for wireless large language model inference in edge computing. FITEE, Springer, 2025. doi:10.1631/FITEE.2400468.

[9] Distributed LLMs and Multimodal Large Language Models: A Survey. 2025. arXiv:2503.16585.

[10] Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. 2025. arXiv:2509.01245.